Experiment No. 10

Mini Project: Study Task Manager

Mobile Application Development – Android

Aim

To design and develop a complete Android application – a Study Task Manager that lets students add, track, complete and delete study tasks stored in a local database.

Apparatus / Software Required

Hardware

- Computer / laptop (min. 8 GB RAM)
- Android phone or emulator

Software

- Android Studio (latest stable) with Android SDK
- Java (JDK 17 bundled with Android Studio)
- SQLite (built into Android) and the RecyclerView library

Theory

A complete Android application combines several concepts: multiple Activities connected by Intents, a data layer to store information permanently, and a list UI to display it. This project uses SQLite through SQLiteOpenHelper for local storage, a RecyclerView with a custom Adapter to show the task list efficiently by reusing item views, and Intents to move between the list screen and the Add Task screen.

Features of the app

- Add a task with title, subject, due date (date picker) and priority.
- View all tasks in a list, pending tasks first.
- Mark a task as done (shown with strike-through) or undo it.
- Delete a task after a confirmation dialog.
- Live summary of pending and total tasks; data stays saved after closing the app.

Implementation Steps

Step 1: Creating the project

Open Android Studio → New Project → Empty Views Activity. Name: StudyTaskManager, package: com.example.studytaskmanager, language: Java, minimum SDK: API 24.

Step 2: Database and project setup

- Add the RecyclerView dependency in app/build.gradle and sync.
- Create the Task model class and DBHelper (SQLiteOpenHelper) with a tasks table.
- Create AddTaskActivity and register it in the manifest.

Column	Type	Purpose
id	INTEGER (PK, auto)	Unique task id

title	TEXT	Task name
subject	TEXT	Subject / course
due_date	TEXT	Due date (dd/mm/yyyy)
priority	TEXT	Low / Medium / High
done	INTEGER	0 = pending, 1 = completed

Step 3: Designing the UI

activity_main.xml has a summary bar, a RecyclerView with an empty-state message, and an Add Task button. item_task.xml defines one row (checkbox, title, details, delete button). activity_add_task.xml has two EditTexts, a date button, a RadioGroup for priority and a Save button.

Step 4: Adding functionality

- DBHelper: methods to add, read, update (done) and delete tasks.
- TaskAdapter: binds each Task to a row and reports checkbox and delete clicks through a listener interface.
- MainActivity: loads tasks in onResume() so the list refreshes after adding; handles toggle and delete.
- AddTaskActivity: validates input, shows a DatePickerDialog, saves the task and closes with finish().

Step 5: Running the application

Run the app, tap Add Task, fill in the details and save. The task appears in the list. Tick the checkbox to complete it, use Delete to remove it, and restart the app to confirm the data is still there.

Files Used / Code Changes

File	Folder	Description
MainActivity.java	java	Home screen: loads tasks, handles toggle/delete, opens Add Task screen.
AddTaskActivity.java	java	Input form with validation and date picker; saves to database.
TaskAdapter.java	java	RecyclerView adapter that fills each list row.
DBHelper.java	java	SQLite database creation and all database operations.
Task.java	java	Model class holding one task's data.
activity_main.xml, activity_add_task.xml, item_task.xml	xml	Layouts of the two screens and one list row.
AndroidManifest.xml	xml	Declares both activities.
strings.xml	xml	Stores all text labels.
build.gradle (notes)	gradle	RecyclerView dependency to add.

Components Used

- Activities (MainActivity, AddTaskActivity) and explicit Intent
- SQLiteOpenHelper / SQLiteDatabase and Cursor

- RecyclerView, LayoutManager and custom Adapter with ViewHolder
- DatePickerDialog and AlertDialog
- EditText, RadioGroup, CheckBox, Button, TextView, Toast

Important XML Attributes Used

Attribute	Purpose
android:id	Unique id to access a view in Java
layout_weight	Gives the list all remaining screen space
android:hint / inputType	Hint text and keyboard type of input fields
android:checked	Sets the default selected priority (Medium)
android:parentActivityName	Parent screen for back navigation
android:gravity / layout_gravity	Aligns content (centered empty-state message)

Important Java Methods Used

Method	Purpose
onCreate() / onUpgrade()	Create and upgrade the database table
insert() / update() / delete() / rawQuery()	Add, change, remove and read rows
moveToNext() / getString() / getInt()	Read values from a Cursor
onCreateViewHolder() / onBindViewHolder()	Create and fill list rows in the adapter
notifyDataSetChanged()	Refreshes the list after data changes
onResume()	Reloads the list when returning to the screen
DatePickerDialog / AlertDialog.Builder	Pick a date and confirm deletion
startActivity() / finish()	Open the Add Task screen and return to the list

Errors Faced & Troubleshooting

Error	Solution
Unresolved reference to RecyclerView	Add the recyclerview dependency in build.gradle and sync.
List not updated after adding a task	Call loadTasks() in onResume() so it runs when returning.
Wrong checkboxes ticked while scrolling	Remove the old listener, then call setChecked() in onBindViewHolder().
App crashes opening Add Task	Register AddTaskActivity in AndroidManifest.xml.
Old data missing after changing table	Increase DB_VERSION or uninstall the app so onUpgrade() / onCreate() runs.

Output

The home screen shows the summary and the task list, with pending tasks first and completed tasks struck through. Add Task opens the form; after saving, the new task appears in the list. Data remains after restarting the app. (Screens below are illustrative samples; replace them with real screenshots from your device.)

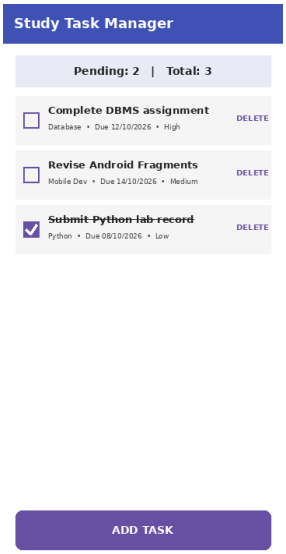


Fig 1: Task list screen

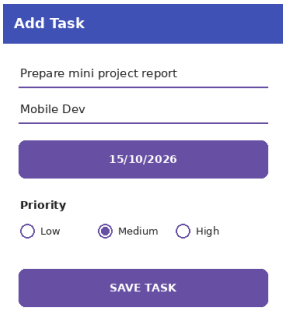


Fig 2: Add Task screen

Conclusion

A complete Android application, the Study Task Manager, was designed and developed using multiple activities, Intents, SQLite storage and a RecyclerView. The mini project combined the concepts learned in the earlier experiments into a working, useful app.

C	A	P	T	SIGN